

Machine Learning for Wind Pressure Coefficient Time Series Forecasting

A Multi-Scale, Multi-Architecture Empirical Study on the TPU BDH Dataset

Cesar Fernando Gamba Tiusaba

Master's Thesis

June 3, 2026

Outline

- 1 Motivation & Objectives
- 2 Data
- 3 Methodology
- 4 Models
- 5 Results
- 6 Discussion
- 7 Conclusions & Future Work

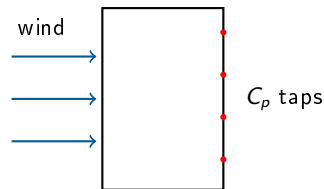
Motivation

Wind pressure coefficients (C_p) on tall buildings govern:

- Cladding design & fatigue life
- Gust-effect factors in codes
- Occupant comfort under wind-induced vibration

Wind tunnels deliver high-frequency C_p time series.

Goal: Can ML/DL forecast C_p *ahead in time* without ground-truth history?



Research Question

Central question

Given 100 ms of past C_p context, can a learning model produce **useful** C_p **forecasts** at lead times from 1 ms to 500 ms, and *what is the limiting factor as the horizon grows?*

Approach in three layers:

- 1 Fair multi-model baseline benchmark ($R1 \rightarrow R2 \rightarrow R3$)
- 2 Intervention on the training objective (direct multi-step)
- 3 Diagnostic spectral analysis + generative counter-example

Two evaluation paradigms (kept distinct):

- **Interpolation** — one-step ahead with known history (gap-filling)
- **True forecasting** — recursive prediction with own outputs (deployment)

Data: TPU BDH Aerodynamic Database

- Sampling: **1000 Hz**
- Series length: **32,768** timesteps (≈ 32.8 s)
- 4 face channels: windward / leeward / sideleft / sideright
- Terrain profiles: $\alpha = 1/4$ and $1/6$
- Aspect ratios (B:D:H): 12 in $\alpha 1/4$, 8 in $\alpha 1/6$
- Yaw angles: 0° – 50° or 0° – 100° (step 5°)
- **Total: 340 series**

Property	Value
Channels per series	4
Window length w	100
Window stride	10
Forecast horizons	1, 10, 50, 100, 500
Train / Val / Test	70 / 15 / 15 %
Normalisation	z-score (train only)
Seed	42

Unified Experimental Protocol

Identical for every model, including classical, RNN, CNN, and Transformer:

Setting	Value
Optimiser	Adam, $lr = 10^{-3}$, weight decay 10^{-4}
Scheduler	ReduceLROnPlateau (patience 5, factor 0.5)
Early stopping	patience 15 on validation loss
Batch size	256
Loss (default)	MSE
Hardware	NVIDIA RTX A4000, CUDA 11.8, PyTorch 2.7.1
Metrics	RMSE, MAE, R^2 , MAPE, directional accuracy

Why a unified protocol?

Any performance difference is attributable to the *model*, not to tuning luck.

Three Rounds of Training Scope

Round	Scope	Geometries	Series	Train windows	Question
R1	1 building	1 (Alpha1_4 / 2_1_3)	17	38 920	Can it learn 1 building?
R2	5 configs	3 ratios $\times$ 2 terrains	85	194 140	Generalises across geometry?
R3	Universal	20 configs $\times$ all yaws	340	776 560	Universal model?

- Same split, same seed, same hyperparameters across rounds.
- Lets us **isolate the effect of data scale** on every pathology.

Interpolation vs True Forecasting

Interpolation (one-step, known history)

$$\hat{y}_t = f(y_{t-n}, \dots, y_{t-1})$$

- Uses **actual** past values
- No error accumulation
- Typical $R^2 \approx 0.99$
- Use case: gap-filling

True forecasting (recursive)

$$\hat{y}_{t+k} = f(\hat{y}_{t+k-n}, \dots, \hat{y}_{t+k-1})$$

- Uses model's **own predictions**
- Errors accumulate
- Typical $R^2 < 0.5$ for long h
- Use case: real-time prediction

Key takeaway

The high $R^2 > 0.98$ from ML papers usually means **interpolation**, not forecasting. We report *both*.

Model Zoo

Family	Model	Role
Baseline	Naive Persistence	Lower bound
	Moving Average, Exp. Smoothing	Smooth-trend assumption
Classical ML	Linear / Ridge / Lasso Regression	Linear ceiling
	Random Forest	Non-linear without temporal order
	XGBoost	Strong boosted-tree baseline
DL autoreg	LSTM, GRU	Gated recurrence
	TCN	Dilated causal convolutions
	CNN-LSTM, Transformer-autoreg	Hybrid / attention
DL direct	LSTM-direct	Same backbone, head = Linear(d,500)
	PatchTST	Patched self-attention
Generative	WPTSE (Tong 2024, replicated)	Stats-conditioned synthesiser

All implementations live in

Agent_Test/<model>/ with a shared data_loader.py and train_utils.py.

AI-Assisted Workflow (Disclosure)

Custom **GitHub Copilot agent** (`wind-cp-forecaster.agent.md`) acted as a *tightly scoped engineering assistant*:

- Enforced the unified protocol (frozen data loader, optimiser, seed)
- Generated boilerplate `train_<model>.py` from a shared spec
- Kept artefacts (CSVs, `.npy` forecasts, plots) deterministically named

Author kept all judgement decisions:

- Which models to include, which horizons, which rounds
- Designing the DM test, PSD diagnostic, WPTSE replication
- Interpreting the spectral collapse and writing the discussion

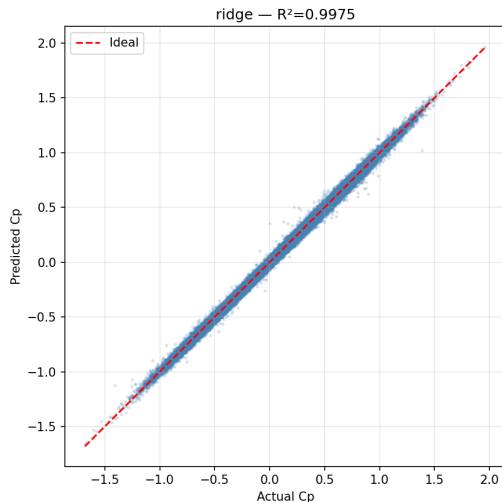
Reporting honesty

The agent did not produce any of the reported numbers. Every metric, table and figure is the output of an executed script committed to the repository.

Pilot Study — Interpolation on a Single Series

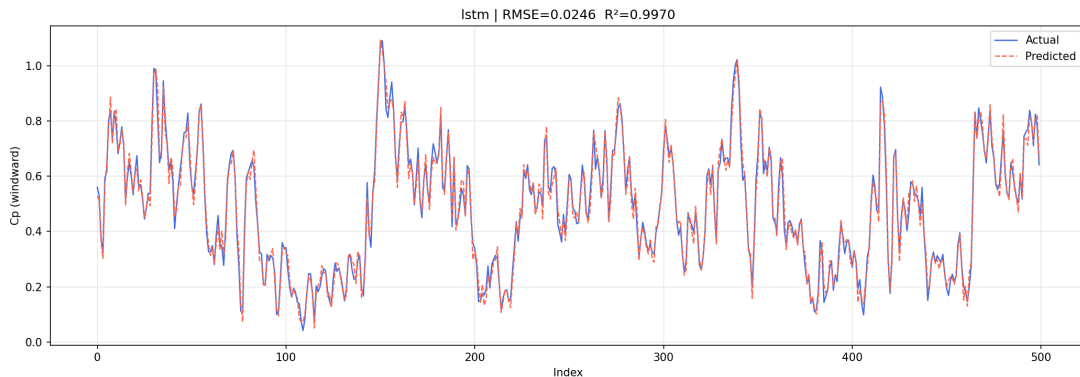
Model	RMSE	R^2
Ridge Regression	0.0391	0.9893
Linear Regression	0.0392	0.9892
Random Forest	0.0411	0.9882
Naive Persistence	0.0473	0.9843
Lasso	0.0484	0.9836
Moving Average $w = 20$	0.1621	0.8157
Simple Exp. Smoothing	0.4551	-0.453
Holt-Winters	0.8381	-3.927

- Linear models saturate $R^2 \approx 0.99$.
- Smooth-trend methods **fail**.
- Naive persistence is strong $\Rightarrow$ high autocorrelation.



Ridge: predicted vs. actual C_p ($R^2 = 0.9975$, R3 single-step)

Single-Step Interpolation in the Time Domain



LSTM at $h=1$ on Round 3 test set — predicted (orange) tracks actual (blue) almost perfectly, $R^2 = 0.997$.

Reading

This is the **interpolation** regime: the model is fed the **true** 100-step context. Useful for gap-filling, *not* a measure of forecasting skill.

Round 3 — Single-Step is Essentially Solved

Model	RMSE	MAE	R^2	Train time (s)
Ridge	0.0224	0.0173	0.9975	1.0
Random Forest	0.0241	0.0185	0.9971	7 262.2
XGBoost	0.0242	0.0182	0.9971	69.0
LSTM	0.0246	0.0188	0.9970	2 730.9
GRU	0.0245	0.0188	0.9970	2 259.9
TCN	0.0186	0.0142	0.9983	8 776.5

- Every reasonable model reaches $R^2 \approx 0.997$.
- Single-step is **not** the differentiator.
- TCN wins by a slim margin — but watch what happens at long horizons.

Autoregressive Rollout Collapses at $h = 500$ (Round 3)

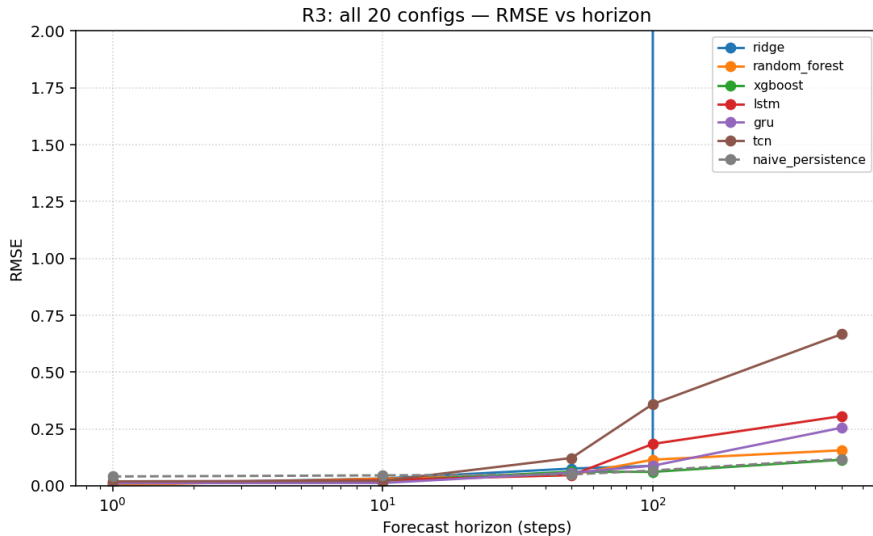
Model	RMSE (autoregressive forecasting)				
	$h=1$	$h=10$	$h=50$	$h=100$	$h=500$
Naive persistence	0.041	0.046	0.050	0.067	0.118
Ridge	0.007	0.032	0.076	0.088	$\sim 10^4$
Random Forest	0.002	0.032	0.048	0.115	0.156
XGBoost	0.011	0.022	0.062	0.061	0.115
LSTM	0.015	0.025	0.047	0.184	0.307
GRU	0.012	0.013	0.057	0.088	0.255
TCN	0.019	0.022	0.122	0.359	0.667

Headline

No autoregressive model beats naive persistence at $h = 500$.

Scaling from R1 (17 series) to R3 (340 series) *does not fix it*. The collapse is **structural**.

Visualising the Collapse — RMSE vs Forecast Horizon



Direct Multi-Step Restores Long-Horizon Skill

Same LSTM backbone — only the head and the loss change:

h	RMSE		R^2	
	direct	autoreg	direct	autoreg
1	0.040	0.025	0.992	0.997
100	0.166	0.184	0.864	-8.06
500	0.179	0.307	0.842	-6.18

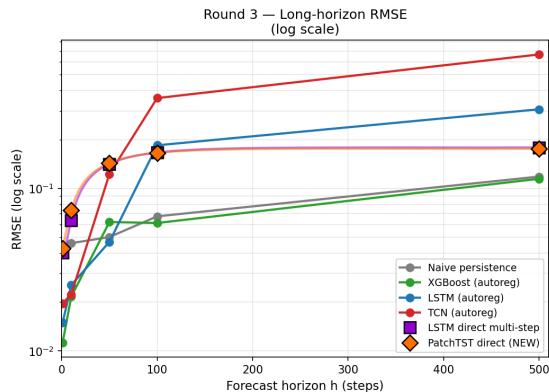
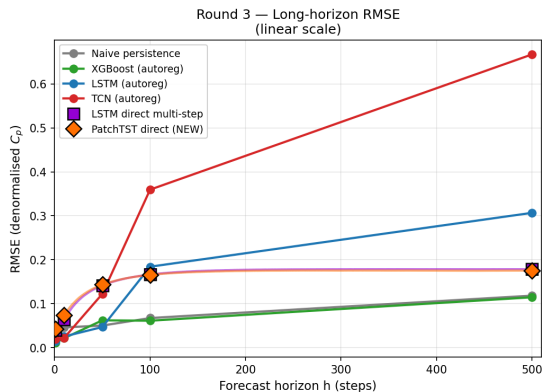
- Switching only the **objective** lifts $h = 500$ R^2 from -6.18 to $+0.84$.
- PatchTST under the same recipe lands at RMSE 0.175 / R^2 0.848 — **the same ceiling**, despite radically different inductive bias.

Conclusion

Long-horizon collapse is an **objective-function problem**, not architectural.

Direct vs Autoregressive — Visual Comparison

Direct multi-step vs autoregressive — wind C_p forecasting



Left: linear scale. **Right:** log scale. LSTM-direct (purple) and PatchTST-direct (orange) converge to the **same RMSE ≈ 0.18** ceiling, while autoregressive TCN/LSTM diverge.

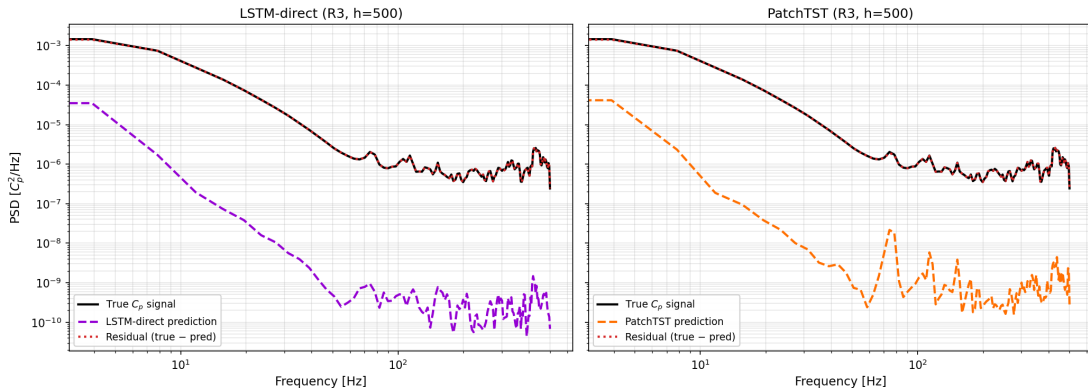
Diebold–Mariano Test (R3, $h = 500$, HAC variance)

Model A	Model B	DM	p -value	Sig.	Winner
Naive	XGBoost	+1.28	0.20	no	tie
Naive	Random Forest	-4.69	$< 10^{-3}$	yes	Naive
Naive	GRU	-5.04	$< 10^{-3}$	yes	Naive
Naive	LSTM	-9.38	$< 10^{-3}$	yes	Naive
Naive	TCN	-15.08	$< 10^{-3}$	yes	Naive
GRU	LSTM	+5.29	$< 10^{-3}$	yes	GRU
LSTM	TCN	-16.58	$< 10^{-3}$	yes	LSTM

- Naive vs. XGBoost is a **statistical tie** at $h = 500$.
- Naive significantly beats every deep autoregressive model.
- The correct long-horizon comparison is *any model vs. persistence*.

Spectral Analysis — The Real Story (Welch PSD)

Power spectral density — wind C_p predictions vs. residuals (Round 3, direct multi-step)



Predictions (dashed) sit $\sim 60\text{--}75\times$ below the true PSD (black) across all frequencies. Residual PSD (red dotted) overlaps the true PSD — the models leak almost all the spectrum into the error.

Quantifying the Spectral Collapse

Quantity	LSTM-direct	PatchTST
Power of true C_p	1.19×10^{-2}	1.19×10^{-2}
Power of predictions	1.58×10^{-4}	1.89×10^{-4}
Ratio $P_{\text{pred}}/P_{\text{true}}$	1.3 % ($\sim 75\times$ smaller)	1.6 % ($\sim 63\times$ smaller)
Power of residuals	1.17×10^{-2}	1.17×10^{-2}

What the models actually output

At $h = 500$ both models predict the **conditional mean** $\mathbb{E}[C_p(t + 500) \mid \mathbf{x}(t - 99 : t)]$.

$R^2 = 0.85$ is **misleading** for wind engineering: fatigue, peak factor and gust-effect factor depend on the *spectrum*, not the conditional mean.

WPTSE — A Generative Counter-Example

Replication of Tong (2024). Same data, but trained as a *synthesiser* (Huber loss, conditioned on

	Model	Task	$P_{\text{pred}}/P_{\text{true}}$	R^2
$\mu, \sigma, \text{skew}, \text{kurt})$:	LSTM-direct	forecast, $h = 500$	1.3 %	0.842
	PatchTST	forecast, $h = 500$	1.6 %	0.848
	WPTSE	synthesis	78.6 %	0.893

- Recovers $\sim 79\%$ of the true spectral power.
- **Nine times fewer** parameters than PatchTST (122 k vs. 1.11 M).
- Relative error in moments: mean 0.7%, kurtosis 3.6%, variance 14%, skew 23%.

Diagnosis

The bound is in the **loss function**, not the architecture.

Conclusions

- 1 Excellent **interpolation** performance with linear models ($R^2 > 0.989$) — ideal for gap-filling.
- 2 **Autoregressive forecasting collapses** at long horizons, *independently* of architecture and data scale.
- 3 The collapse is the **conditional-mean attractor** of MSE training on a chaotic signal, confirmed by two architectures (LSTM, PatchTST) and by spectral analysis.

- Distributional / generative forecasting:
 - Quantile regression with pinball loss
 - Diffusion-based forecasters
 - Add some time-dependent function deviation to obligate the models to capture temporal dynamics
- Complete TFT training at R3 scope (deferred for compute budget).
- Foundation models for time series: TimesFM, Chronos, Lag-Llama (maybe).
- Spectral-fidelity metrics adopted as standard reporting alongside RMSE/ R^2 .
- Cross-sensor / cross-building transfer learning via encoder representations.
- API and Infrastructure for deployment Streamlit.

Thank you

Repository: Wind_BWU/Agent_Test/
Wind_ML_TimeSeries/Thesis/Latex_Document/main.pdf